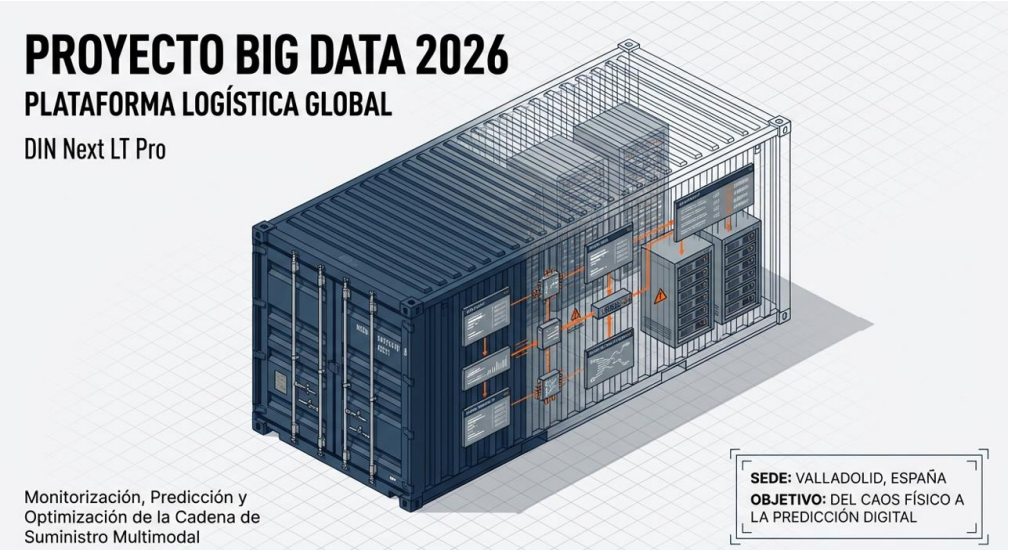

Ana Coloma Bausela

Confidential C

Entorno: **Docker/local sobre Linux**, con `docker compose` para la demo completa y modo `dashboard solo (ultraligero)` para iteración visual sin levantar todo el stack.

Estado real de implementación (checkpoint)

- Stack validado en la ruta oficial Docker/local:
  - HDFS (`namenode` + `datanode`)
  - Kafka KRaft
  - NiFi 2.6.0
  - Spark 3.5.x + Hive + GraphFrames
  - Cassandra 5.0
  - Airflow 2.10.5
  - Streamlit para la defensa funcional
- Jobs KDD implementados:
  - `jobs/spark/01_raw_to_staging.py`
  - `jobs/spark/01_load_master_dimensions.py`
  - `jobs/spark/01_weather_filtered_to_staging.py`
  - `jobs/spark/02_weather_port_enrichment.py`
  - `jobs/spark/03_weather_operational_fact.py`
  - `jobs/spark/02_graph_metrics.py`
  - `jobs/spark/03_score_and_alert.py`
  - `jobs/spark/03_air_recovery_options.py`
  - `jobs/spark/99_dashboard_bundle.py`
- Tablas de salida verificadas:
  - `logistica.stg_ships`, `logistica.stg_alerts_clima`, `logistica.stg_alerts_noticias`, `logistica.stg_weather_open_meteo`
  - `logistica.dim_ports`, `logistica.dim_routes`, `logistica.dim_warehouse`, `logistica.dim_skus`
  - `logistica.dim_ports_routes_weather`, `logistica.fact_weather_operational`
  - `logistica.fact_route_risk`, `logistica.fact_graph_hops`, `logistica.fact_graph_centrality`, `logistica.fact_alerts`
  - `logistica.fact_air_recovery_options`, `logistica.dim_articles_valladolid`, `logistica.fact_customer_orders_douai`, `logistica.fact_article_gantt`
  - `logistica.vehicle_latest_state` en Cassandra

Notas operativas ya confirmadas:

- La ruta recomendada para la defensa es Docker/local, no VirtualBox.

- El dashboard puede arrancar sin Docker con `./start.sh` opción `3` o `bash scripts/67_run_dashboard.sh`. - Si falla una regeneración del bundle, el dashboard reutiliza `jobs/dashboard_bundle_output.last_good.json`.

- Las alertas críticas por correo no bloquean la UI: si SMTP no es accesible, se guarda evidencia demo en `artifacts/email_alerts/`.

Este estado de implementación permite defender el proyecto como una solución funcional de extremo a extremo, con una capa de visualización ejecutiva adaptada a la exposición académica.

1. Contexto y problema (KDD – Selección)

1.1 Sector y caso de uso

El proyecto se sitúa en el contexto de la logística internacional de componentes, con especial foco en la importación marítima desde Asia hacia España y la distribución final al almacén de Valladolid.

Los puntos críticos del caso de uso son los siguientes: - nivel de stock disponible y riesgo de ruptura en el almacén - tiempos de tránsito desde el puerto de origen hasta el destino final

- demoras operativas en descarga, gestión portuaria y preparación de expediciones

- impacto de factores externos, especialmente clima adverso y contexto geopolítico

1.2 Objetivo del sistema

El objetivo del sistema es monitorizar la situación logística de la cadena de suministro mediante datos de posición, eventos operativos y señales externas de riesgo. A partir de esta información, la plataforma debe identificar incidencias relevantes y traducirlas en alertas accionables, incluyendo la activación de planes de contingencia cuando la continuidad del suministro esté comprometida.

1.3 Justificación de decisiones

Las principales decisiones tecnológicas se justifican del siguiente modo:

- **`Kafka`** desacopla las fuentes de datos y los consumidores analíticos.

- **`HDFS`** proporciona persistencia raw auditable y una base escalable para staging y curated.

- **`Spark`** resuelve la limpieza, tipado, enriquecimiento y construcción de facts analíticas. Para maximizar el rendimiento, se ha implementado una estrategia de **Broadcast Hash Joins**. Esta técnica es ideal para nuestra arquitectura de clúster, ya que las tablas de dimensiones como dim_ports y dim_warehouse son de tamaño reducido.

Al marcar estas tablas para *broadcast*, Spark las distribuye íntegramente en la memoria de cada ejecutor antes de iniciar la transformación. Esto permite que el join con el flujo masivo de datos de barcos (stg_ships) se realice de forma local en cada nodo, eliminando el **Shuffle** (intercambio de datos por red). El resultado es una reducción drástica en la latencia del pipeline y una ejecución más estable en nuestro entorno de despliegue basado en contenedores.

- **`Hive`** actúa como capa SQL histórica y defendible para consulta de resultados.

- **`GraphFrames`** permite modelar la red logística como grafo y calcular criticidad estructural.

- **`Cassandra`** actúa como la capa de servicio (Serving Layer) de baja latencia para el Dashboard, cubriendo el requisito de consulta rápida sobre el último estado operativo de la flota en contraposición a las consultas analíticas pesadas de Hive.

- **`Airflow`** aporta orquestación, dependencias, reintentos y trazabilidad de ejecución.

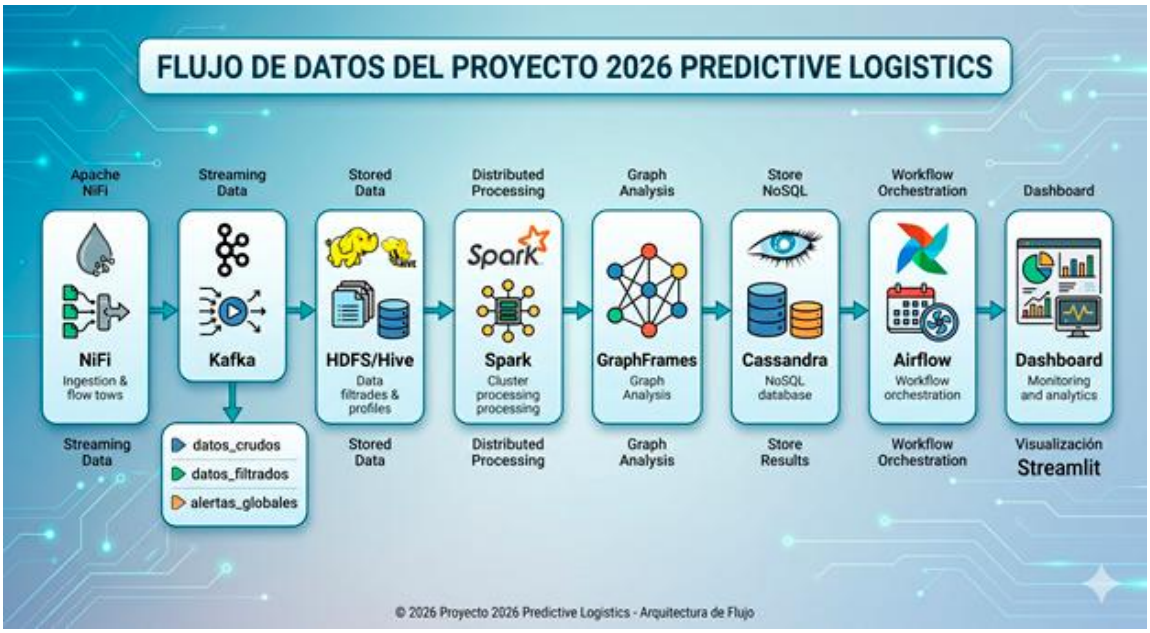

***Figura 1**: Arquitectura general de la solución logística basada en NiFi, Kafka, HDFS, Spark, Hive, Cassandra, Airflow y Streamlit.*

2. Arquitectura final (KDD – Selección + Preprocesamiento)

2.1 Arquitectura (Lambda/Kappa adaptada)

La defensa se apoya en una ruta Docker/local reproducible. La estrategia principal es **micro-batch documentado**. Como evidencia técnica complementaria, se ha implementado el job `04_streaming_ml_pipeline.py` utilizando **Structured Streaming**.

Este componente permite procesar eventos mediante ventanas de tiempo (windowing) de 15 minutos, calculando medias móviles de retrasos y estados operativos. Aunque la defensa se centra en el modelo micro-batch para estabilidad de la demo, esta implementación demuestra la capacidad del sistema para operar bajo una arquitectura Kappa pura

Solución final defendida: - Ingesta real con NiFi hacia Kafka (`datos_crudos`, `datos_filtrados`, `alertas_globales`). - Persistencia **raw/staging/curated** en HDFS y Hive.

- Persistencia operativa adicional en Cassandra. - Exposición ejecutiva y técnica mediante dashboard Streamlit.

2.2 Componentes (ruta oficial Docker/local)

La solución final se articula sobre los siguientes componentes:

- `namenode` y `datanode`: persistencia distribuida en HDFS

- `kafka`: mensajería de eventos sobre KRaft - `nifi`: adquisición, filtrado y publicación inicial de datos

- `spark`: ejecución de jobs batch, enriquecimiento, facts y análisis de grafos

- `cassandra`: consulta de estado actual con baja latencia

- `airflow-webserver`: supervisión y orquestación de workflows

- `dashboard` local en `Streamlit`: visualización ejecutiva y técnica para la defensa

```
$ docker compose ps

NAME                          IMAGE                                          COMMAND                  SERVICE             CREATED          STATUS
                      PORTS
logistica-airflow-webserver-1   apache/airflow:2.10.5-python3.10             "/usr/bin/dumb-init …"   airflow-webserver   47 seconds ago   Up 12
seconds                       0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp
logistica-datanode-1            bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8   "/entrypoint.sh /run…"   datanode            47 seconds ago   Up 15
seconds (health: starting)    9864/tcp
logistica-kafka-1               apache/kafka:3.9.1                           "/__cacert_entrypoin…"   kafka               47 seconds ago   Up 46
seconds (healthy)             9092/tcp, 0.0.0.0:9092->9094/tcp, [::]:9092->9094/tcp
logistica-namenode-1            bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8   "/entrypoint.sh /run…"   namenode            47 seconds ago   Up 46
seconds (healthy)             0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp, 0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp
logistica-nifi-1                apache/nifi:2.6.0                            "../scripts/start.sh"    nifi                47 seconds ago   Up 46
seconds                       8000/tcp, 10000/tcp, 0.0.0.0:8443->8443/tcp, [::]:8443->8443/tcp
logistica-postgres-1            postgres:13                                  "docker-entrypoint.s…"   postgres            47 seconds ago   Up 46
seconds                       5432/tcp
logistica-spark-1               jupyter/pyspark-notebook:latest              "tini -g -- start-no…"   spark               47 seconds ago   Up 12
seconds (healthy)             0.0.0.0:4040->4040/tcp, [::]:4040->4040/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:8888->8888/tcp, [::]:8888-
>8888/tcp
```

Figura 2. Estado de los servicios principales de la plataforma en el entorno Docker/local.

.3. Preparación del entorno (Docker/local)

3.1 Recursos y perfiles de arranque

El proyecto dispone de dos perfiles de ejecución complementarios:

- Ruta completa de demostración, orientada a la validación integral del stack: `docker compose up -d`
- Ruta ligera de trabajo visual, orientada a iterar sobre el dashboard sin penalización de recursos: `./start.sh` y selección de la opción 3

Cuando se requiere regenerar el dashboard con datos reales, la secuencia validada es la siguiente: - `bash scripts/66_rebuild_hive_demo_tables.sh` - `docker compose exec -T spark spark-submit /home/jovyan/jobs/spark/99_dashboard_bundle.py` - `bash scripts/67_run_dashboard.sh`

```
ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ bash scripts/60_manage_docker_stack.sh start simple core
Repositorio: /home/ana/Descargas/PROYECTOLOGISTICA_codigo/Logistica
Compose: docker-compose.simple.yml
Perfil: core
Fase 1/3: servicios base
logistica_nifi_1 is up-to-date
logistica_kafka_1 is up-to-date
logistica_postgres_1 is up-to-date
logistica_airflow_1 is up-to-date
Inicializando topics Kafka
logistica_kafka_1 is up-to-date
Starting logistica_kafka-init_1 ... done
```

***Figura 3**. Servicios principales levantados en la ruta Docker/local para la validación integral del entorno.*

3.2 Regla crítica de operación

Durante las validaciones se identificó una restricción operativa relevante: los comandos `spark-sql` y `spark-submit` que usan Hive deben ejecutarse **de forma secuencial**, ya que la ejecución paralela puede bloquear el metastore embebido basado en Derby.

Asimismo, si HDFS entra en `safe mode` o la regeneración del bundle falla, el dashboard puede quedarse sin datos. Para mitigar ese riesgo, la aplicación conserva un último bundle válido en `jobs/dashboard_bundle_output.last_good.json` y lo reutiliza automáticamente cuando el nuevo fichero no aporta datos útiles.

4. Ingesta (KDD – Ingesta y Selección): Kafka KRaft + “landing raw” HDFS

4.1 Topics Kafka

- `datos_crudos`: GPS barcos, eventos operativos (puerto, almacén, stock).
- `alertas_globales`: noticias geopolíticas y clima (y/o eventos agregados).

```
$ docker-compose -f docker-compose.simple.yml exec -T kafka /opt/kafka/
bin/kafka-console-consumer.sh --bootstrap-server kafka:9092 --topic dat
os_filtrados --from-beginning --max-messages 3
{"ts":"2026-03-28T12:15","temperature_c":16.3,"humidity_pct":69,"wind_s
peed_kmh":18.0,"wind_direction_deg":92,"weather_code":2,"region":"Estre
cho de Gibraltar","source":"open_meteo","event_type":"weather_snapshot"
,"port_ref":"Algeciras"}
{"ts":"2026-03-28T12:15","temperature_c":16.3,"humidity_pct":69,"wind_s
peed_kmh":18.0,"wind_direction_deg":92,"weather_code":2,"region":"Estre
cho de Gibraltar","source":"open_meteo","event_type":"weather_snapshot"
,"port_ref":"Algeciras"}
{"ts":"2026-03-28T12:15","temperature_c":16.3,"humidity_pct":69,"wind_s
peed_kmh":18.0,"wind_direction_deg":92,"weather_code":2,"region":"Estre
cho de Gibraltar","source":"open_meteo","event_type":"weather_snapshot"
,"port_ref":"Algeciras"}
Processed a total of 3 messages
```

***Figura 4**. Topics Kafka creados para la ingesta y el procesamiento.*

```
ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker compose exec -T kafka bash -lc "printf '%s\n' '{\"ship_id\":\"ship-001\",\"ts\":\"2026-04-17T12:00:00\",\"lat\":36.0,\"
lon\":-5.0}' | /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server kafka:9092 --topic datos_crudos"
ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker compose exec -T kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka:9092 --topic datos_crudos --fro
m-beginning --max-messages 5
{"ship_id":"ship-001","ts":"2026-04-17T12:00:00","lat":36.0,"lon":-5.0}
```

```
^C
ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker compose exec -T kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --describe --topic datos_crudos
Topic: datos_crudos     TopicId: qYEJG0XwRMe6R9K-1z9KPw PartitionCount: 1       ReplicationFactor: 1    Configs:
        Topic: datos_crudos     Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
```

Figura 5: Evidencias de consumo de eventos en el topic datos_crudos.

4.2 Evidencia del flujo NiFi

- Flujo exportado reutilizable:
  - `docs/nifi/OpenMeteo_Kafka_Flow.json`
- En la memoria, este artefacto se utiliza para:
  - referenciar el JSON como evidencia técnica del canvas real
  - acompañarlo de una captura PNG del flujo en ejecución
  - justificar que el export permite reimportar, auditar y reproducir la ingesta

4.2 Formato de eventos (contratos)

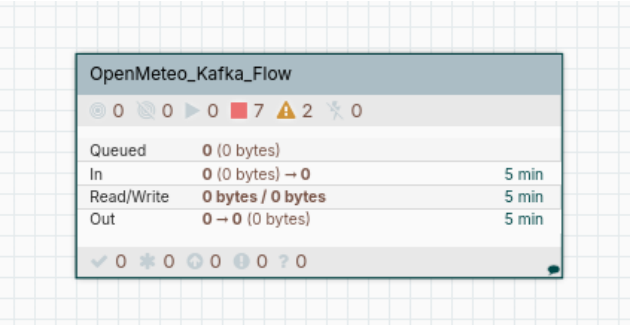

Figura 6. Visión general del grupo de proceso de NiFi.

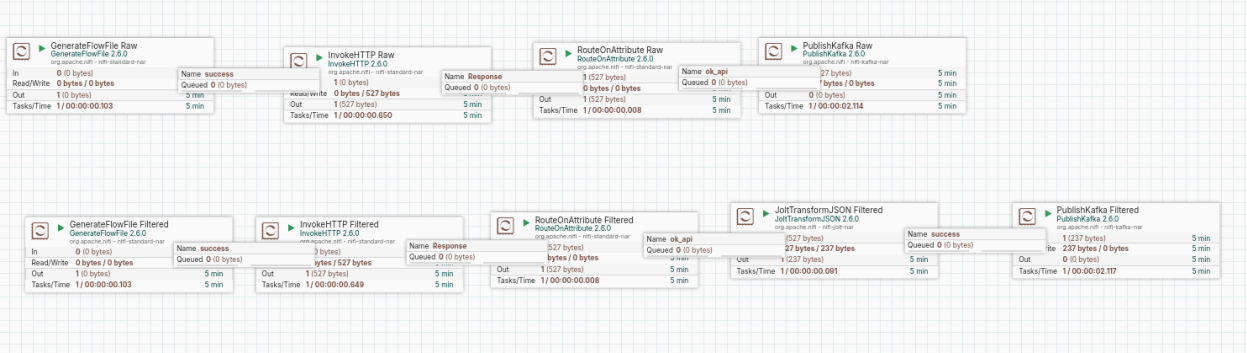

Figura 7. Canvas completo de NiFi para la ingesta meteorológica y publicación en Kafka.

Se propone documentar un contrato JSON mínimo para cada tipo de evento:

- **ShipPositionEvent**: `ship_id`, `ts`, `lat`, `lon`, `speed`, `heading`, `route_id`, `eta_port`

- **AlertEvent**: `source` (clima/noticias), `ts`, `severity`, `region`, `text`, `confidence`

- **StockEvent**: `sku`, `warehouse` (Valladolid), `ts`, `stock_on_hand`, `reorder_point`

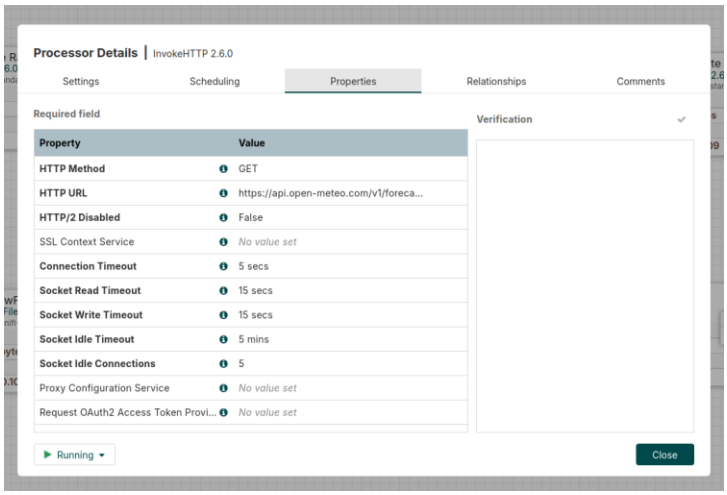

Figura 8. *Configuración del procesador InvokeHTTP en NiFi*.

Justificación: contratos estables y explícitos para desacoplar la ingesta de las fases analíticas posteriores.

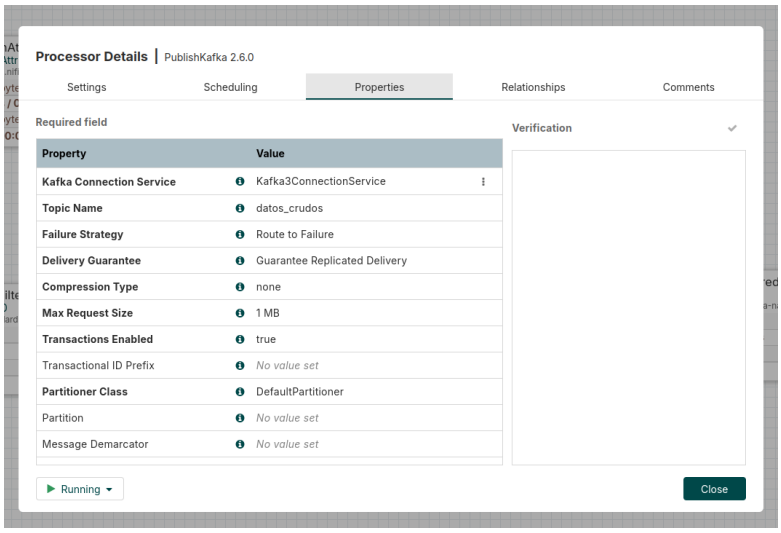

Figura 9. Configuración del procesador PublishKafka en NiFi

```
ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ bash scripts/61_nifi_healthcheck.sh
Comprobando NiFi en https://localhost:8443
Token recibido correctamente
NiFi name: NiFi
NiFi version: 2.6.0
```

Figura 10. Flujo NiFi en ejecución con procesamiento correcto.

4.3 Landing raw en HDFS

Rutas: - `/hadoop/logistica/raw/ships` - `/user/logistica/raw/opensky` `/hadoop/logistica/raw/noticias` - `/hadoop/logistica/raw/clima`

```
ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker compose exec -T namenode bash -lc '/opt/hadoop-3.2.1/bin/hdfs dfs -ls -R /hadoop/logistica/raw'
mesg: ttyname failed: Inappropriate ioctl for device
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/clima
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/clima/2026
drwxr-xr-x   - root supergroup          0 2026-03-28 21:33 /hadoop/logistica/raw/clima/2026/03
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/clima/2026/03/26
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/clima/2026/03/26/1840
-rw-r--r--   1 root supergroup        388 2026-03-26 19:15 /hadoop/logistica/raw/clima/2026/03/26/1840/clima_sample.jsonl
drwxr-xr-x   - root supergroup          0 2026-03-28 21:33 /hadoop/logistica/raw/clima/2026/03/28
drwxr-xr-x   - root supergroup          0 2026-03-28 21:41 /hadoop/logistica/raw/clima/2026/03/28/2100
-rw-r--r--   1 root supergroup        388 2026-03-28 21:41 /hadoop/logistica/raw/clima/2026/03/28/2100/clima_sample.jsonl
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/noticias
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/noticias/2026
drwxr-xr-x   - root supergroup          0 2026-03-28 21:33 /hadoop/logistica/raw/noticias/2026/03
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/noticias/2026/03/26
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/noticias/2026/03/26/1840
-rw-r--r--   1 root supergroup        419 2026-03-26 19:15 /hadoop/logistica/raw/noticias/2026/03/26/1840/noticias_sample.jsonl
drwxr-xr-x   - root supergroup          0 2026-03-28 21:33 /hadoop/logistica/raw/noticias/2026/03/28
drwxr-xr-x   - root supergroup          0 2026-03-28 21:41 /hadoop/logistica/raw/noticias/2026/03/28/2100
-rw-r--r--   1 root supergroup        419 2026-03-28 21:41 /hadoop/logistica/raw/noticias/2026/03/28/2100/noticias_sample.jsonl
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/ships
drwxr-xr-x   - root supergroup          0 2026-04-03 20:33 /hadoop/logistica/raw/ships/2026
drwxr-xr-x   - root supergroup          0 2026-03-28 21:33 /hadoop/logistica/raw/ships/2026/03
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/ships/2026/03/26
drwxr-xr-x   - root supergroup          0 2026-03-26 19:15 /hadoop/logistica/raw/ships/2026/03/26/1840
-rw-r--r--   1 root supergroup        679 2026-03-26 19:15 /hadoop/logistica/raw/ships/2026/03/26/1840/ships_sample.jsonl
drwxr-xr-x   - root supergroup          0 2026-03-28 21:33 /hadoop/logistica/raw/ships/2026/03/28
drwxr-xr-x   - root supergroup          0 2026-03-28 21:41 /hadoop/logistica/raw/ships/2026/03/28/2100
-rw-r--r--   1 root supergroup        679 2026-03-28 21:41 /hadoop/logistica/raw/ships/2026/03/28/2100/ships_sample.jsonl
drwxr-xr-x   - root supergroup          0 2026-04-03 20:33 /hadoop/logistica/raw/ships/2026/04
drwxr-xr-x   - root supergroup          0 2026-04-03 20:33 /hadoop/logistica/raw/ships/2026/04/03
drwxr-xr-x   - root supergroup          0 2026-04-03 20:33 /hadoop/logistica/raw/ships/2026/04/03/2030
-rw-r--r--   1 root supergroup       1358 2026-04-03 20:33 /hadoop/logistica/raw/ships/2026/04/03/2030/ships_sample.jsonl
```

Figura 11. *Aterrizaje raw en HDFS para ships, clima y noticias.*

Nota: el mecanismo concreto de copia raw puede resolverse con NiFi o con consumidores Python, siempre que quede evidenciado el aterrizaje auditable en HDFS. En el estado actual del proyecto, este paso ha sido validado con productores Python y un consumidor Kafka->HDFS, manteniendo además la evidencia del flujo NiFi para la memoria.

4.4 Ejecución end-to-end (producción + landing raw)

Para tener evidencia en HDFS rápidamente, ejecutamos: - productor de **GPS barcos** → `datos_crudos` - productor de **alertas globales** (clima/noticias) → `alertas_globales` - consumidor que hace **landing raw** JSONL en HDFS

Paso 1: activar el entorno de mensajería

```
docker compose up -d Kafka
```

Paso 2: crear entorno Python de ingesta

```
cd /home/ana/Descargas/PROYECTOLOGISTICA_codigo/Logistica
python3 -m venv .venv-ingesta
source .venv-ingesta/bin/activate
pip install -r ingesta/requirements.txt
```

Paso 3: lanzar productores (2 terminales)

Terminal A (GPS barcos):

```
source .venv-ingesta/bin/activate
python3 ingesta/productores/ships_gps_producer.py --bootstrap localhost:9092
--topic datos_crudos --ships 10 --interval-sec 0.5
```

Terminal B (alertas clima/noticias):

```
source .venv-ingesta/bin/activate
python3 ingesta/productores/alerts_producer.py --bootstrap localhost:9092 --
topic alertas_globales --interval-sec 1.0
```

Paso 4: lanzar landing raw (3ª terminal)

```
source .venv-ingesta/bin/activate
python3 ingesta/consumidores/kafka_to_hdfs_raw.py --bootstrap localhost:9092
--group-id logistica-raw-sink \
  --spool-dir /tmp/logistica_spool --flush-every-sec 30
```

Paso 5: verificar que se genera la evidencia

```
hdfs dfs -ls -R /hadoop/logistica/raw/ships | head
hdfs dfs -ls -R /hadoop/logistica/raw/clima | head
hdfs dfs -ls -R /hadoop/logistica/raw/noticias | head
```

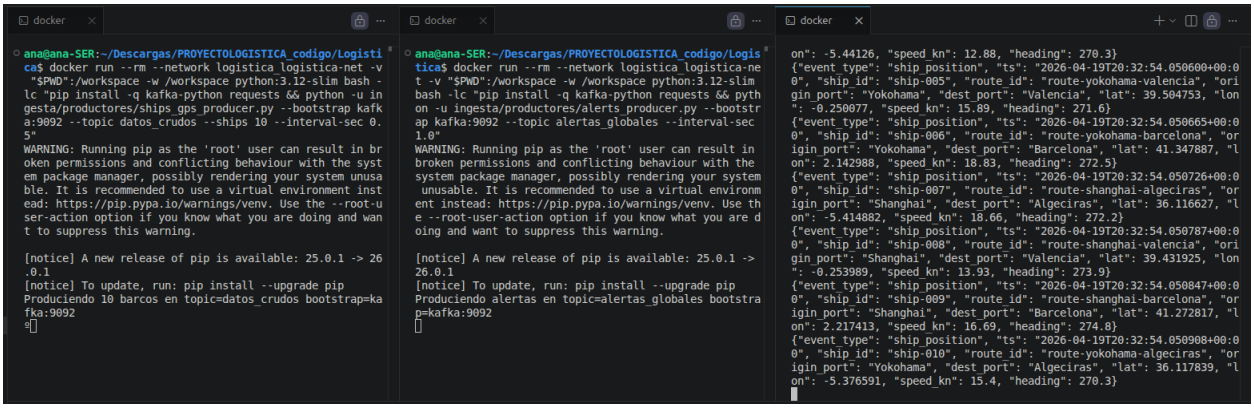

Figura 12. Ejecución end-to-end con productores y consumidor activos.

5. Preprocesamiento y transformación (KDD – Preprocesamiento / Transformación): Spark SQL + Hive

5.1 Limpieza y normalización

- Parsing JSON → columnas tipadas.
- Control de nulos y duplicados.
- Normalización de timestamps/zonas horarias.

```
org.lz4_lz4-java-1.8.0.jar
26/04/17 19:04:04 INFO Executor: Fetching file:///home/jovyan/.ivy2/jars/commons-logging_commons-logging-1.1.3.jar with timestamp 1776452641456
26/04/17 19:04:04 INFO Utils: /home/jovyan/.ivy2/jars/commons-logging_commons-logging-1.1.3.jar has been previously copied to /tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-995
0-ed54f088cd32/commons-logging_commons-logging-1.1.3.jar
26/04/17 19:04:04 INFO Executor: Fetching file:///home/jovyan/.ivy2/jars/org.slf4j_slf4j-api-2.0.7.jar with timestamp 1776452641456
26/04/17 19:04:04 INFO Utils: /home/jovyan/.ivy2/jars/org.slf4j_slf4j-api-2.0.7.jar has been previously copied to /tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd
32/org.slf4j_slf4j-api-2.0.7.jar
26/04/17 19:04:04 INFO Executor: Fetching file:///home/jovyan/.ivy2/jars/org.apache.commons_commons-pool2-2.11.1.jar with timestamp 1776452641456
26/04/17 19:04:04 INFO Utils: /home/jovyan/.ivy2/jars/org.apache.commons_commons-pool2-2.11.1.jar has been previously copied to /tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9
950-ed54f088cd32/org.apache.commons_commons-pool2-2.11.1.jar
26/04/17 19:04:04 INFO Executor: Fetching spark://8031940adaca:40383/jars/org.slf4j_slf4j-api-2.0.7.jar with timestamp 1776452641456
26/04/17 19:04:04 INFO TransportClientFactory: Successfully created connection to 8031940adaca/172.19.0.9:40383 after 102 ms (0 ms spent in bootstraps)
26/04/17 19:04:05 INFO Utils: Fetching spark://8031940adaca:40383/jars/org.slf4j_slf4j-api-2.0.7.jar to /tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/fetchFi
leTemp4975610535785399533.tmp
26/04/17 19:04:05 INFO Utils: /tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/fetchFileTemp4975610535785399533.tmp has been previously copied to /tmp/spark-cf8
68e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/org.slf4j_slf4j-api-2.0.7.jar
26/04/17 19:04:05 INFO Executor: Adding file:/tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/org.slf4j_slf4j-api-2.0.7.jar to class loader default
26/04/17 19:04:05 INFO Executor: Fetching spark://8031940adaca:40383/jars/org.lz4_lz4-java-1.8.0.jar with timestamp 1776452641456
26/04/17 19:04:05 INFO Utils: Fetching spark://8031940adaca:40383/jars/org.lz4_lz4-java-1.8.0.jar to /tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/fetchFileT
emp7728128786219802511.tmp
26/04/17 19:04:05 INFO Utils: /tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/fetchFileTemp7728128786219802511.tmp has been previously copied to /tmp/spark-cf8
68e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/org.lz4_lz4-java-1.8.0.jar
26/04/17 19:04:05 INFO Executor: Adding file:/tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/org.lz4_lz4-java-1.8.0.jar to class loader default
26/04/17 19:04:05 INFO Executor: Fetching spark://8031940adaca:40383/jars/org.apache.spark_spark-sql-kafka-0-10_2.12-3.5.0.jar with timestamp 1776452641456
26/04/17 19:04:05 INFO Utils: Fetching spark://8031940adaca:40383/jars/org.apache.spark_spark-sql-kafka-0-10_2.12-3.5.0.jar to /tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-99
50-ed54f088cd32/fetchFileTemp14393219082716122976.tmp
26/04/17 19:04:05 INFO Utils: /tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/fetchFileTemp14393219082716122976.tmp has been previously copied to /tmp/spark-cf
868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/org.apache.spark_spark-sql-kafka-0-10_2.12-3.5.0.jar
26/04/17 19:04:05 INFO Executor: Adding file:/tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54f088cd32/org.apache.spark_spark-sql-kafka-0-10_2.12-3.5.0.jar to class load
er default
26/04/17 19:04:05 INFO Executor: Fetching spark://8031940adaca:40383/jars/org.apache.hadoop_hadoop-client-api-3.3.4.jar with timestamp 1776452641456
26/04/17 19:04:05 INFO Utils: Fetching spark://8031940adaca:40383/jars/org.apache.hadoop_hadoop-client-api-3.3.4.jar to /tmp/spark-cf868e61-1996-4fab-ac86-e1e003289206/userFiles-3616477b-714d-4e8d-9950-ed54
```

Figura 13. Ejecución de un job Spark de transformación hacia staging.

Ejecución del job 01 (ejemplo local/docker):

```
docker compose exec -T spark spark-submit
/home/jovyan/jobs/spark/01_raw_to_staging.py
```

Ejecución del job meteorológico validado en Docker:

```
bash scripts/63_run_weather_filtered_staging.sh full
```

Comando directo equivalente:

```
docker compose exec -T spark spark-submit --packages org.apache.spark:spark-
sql-kafka-0-10_2.12:3.5.0
/home/jovyan/jobs/spark/01_weather_filtered_to_staging.py --bootstrap
kafka:9092 --topic datos_filtrados
```

Verificación en HDFS (evidencia):

```
hdfs dfs -ls -R /hadoop/logistica/staging/stg_ships | head
hdfs dfs -ls -R /hadoop/logistica/staging/stg_alerts_clima | head
hdfs dfs -ls -R /hadoop/logistica/staging/stg_alerts_noticias | head
```

```
(.venv-jobs) ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker-compose exec -T spark bash -lc 'spark-sql -e "SELECT * FROM logistica.stg_weather_open_meteo ORDER BY event_ts DESC LIMI
T 5"'
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/03/28 12:38:41 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/03/28 12:38:43 WARN HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist
26/03/28 12:38:43 WARN HiveConf: HiveConf of name hive.stats.retries.wait does not exist
26/03/28 12:38:44 WARN ObjectStore: Version information not found in metastore. hive.metastore.schema.verification is not enabled so recording the schema version 2.3.0
26/03/28 12:38:44 WARN ObjectStore: setMetaStoreSchemaVersion called but recording version is disabled: version = 2.3.0, comment = Set by MetaStore UNKNOWN@172.19.0.3
Spark Web UI available at http://446f29ef21cc:4040
Spark master: local[*], Application Id: local-1774701522805
26/03/28 12:38:45 WARN ObjectStore: Failed to get database global_temp, returning NoSuchObjectException
2026-03-28T12:45:00     18.0    61      39.5    110     1       Algeciras    open_meteo       weather_snapshot        Estrecho de Gibraltar   2026-03-28 12:45:00   3       2026-03-28 12:32:08.746532
2026-03-28T12:30:00     17.1    66      24.0    101     2       Algeciras    open_meteo       weather_snapshot        Estrecho de Gibraltar   2026-03-28 12:30:00   2       2026-03-28 12:32:08.746532
2026-03-28T12:15:00     16.3    69      18.0    92      2       Algeciras    open_meteo       weather_snapshot        Estrecho de Gibraltar   2026-03-28 12:15:00   1       2026-03-28 12:32:08.746532
```

Figura 14. Evidencia de la tabla staging stg_weather_open_meteo en Hive.

5.2 Enriquecimiento con datos maestros (Hive)

Tablas maestras (ejemplos): - `dim_ports` (Shanghai, Algeciras/Valencia/Barcelona) - `dim_warehouse` (Valladolid) - `dim_routes` (rutas marítimas y puerto→Valladolid) - `dim_skus` (lead time, criticidad)

```
26/03/28 12:44:09 INFO audit: ugi=jovyan        ip=unknown-ip-addr      cmd=get_tables: db=logistica pat=*
26/03/28 12:44:10 INFO CodeGenerator: Code generated in 23.022254 ms
26/03/28 12:44:10 INFO CodeGenerator: Code generated in 5.703894 ms
26/03/28 12:44:10 INFO CodeGenerator: Code generated in 7.71406 ms
+---------+----------------------+-----------+
|namespace|tableName             |isTemporary|
+---------+----------------------+-----------+
|logistica|dim_ports             |false      |
|logistica|dim_routes            |false      |
|logistica|dim_skus              |false      |
|logistica|dim_warehouse         |false      |
|logistica|stg_weather_open_meteo|false      |
+---------+----------------------+-----------+

26/03/28 12:44:10 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/03/28 12:44:10 INFO SparkUI: Stopped Spark web UI at http://446f29ef21cc:4040
26/03/28 12:44:10 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/03/28 12:44:10 INFO MemoryStore: MemoryStore cleared
```

Figura 15. *Tablas de dimensiones maestras cargadas en Hive.*

```
(.venv-jobs) ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker-compose exec -T spark spark-submit /home/jovyan/jobs/spark/02 weather port enrichment.py
+---------+----------+----------------------+--------+-----------------+----------------------+---------------------------+
|Algeciras|Shanghai  |route-shanghai-algeciras|3       |MEDIO            |METEO_VIGILANCIA      |6.0                        |
|Algeciras|Shanghai  |route-shanghai-algeciras|2       |MEDIO            |OPERATIVO             |3.0                        |
|Algeciras|Shanghai  |route-shanghai-algeciras|1       |BAJO             |OPERATIVO             |1.0                        |
+---------+----------+----------------------+--------+-----------------+----------------------+---------------------------+

26/03/28 12:48:38 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/03/28 12:48:38 INFO SparkUI: Stopped Spark web UI at http://446f29ef21cc:4040
26/03/28 12:48:38 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/03/28 12:48:38 INFO MemoryStore: MemoryStore cleared
26/03/28 12:48:38 INFO BlockManager: BlockManager stopped
26/03/28 12:48:38 INFO BlockManagerMaster: BlockManagerMaster stopped
26/03/28 12:48:38 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/03/28 12:48:38 INFO SparkContext: Successfully stopped SparkContext
26/03/28 12:48:39 INFO ShutdownHookManager: Shutdown hook called
26/03/28 12:48:39 INFO ShutdownHookManager: Deleting directory /tmp/spark-4d739e62-8cdc-4cde-ac32-ec7cfafc9fea
26/03/28 12:48:39 INFO ShutdownHookManager: Deleting directory /tmp/spark-ce037936-0259-46b1-b1e2-6f8173871a2c
26/03/28 12:48:39 INFO ShutdownHookManager: Deleting directory /tmp/spark-ce037936-0259-46b1-b1e2-6f8173871a2c/pyspark-f9872975-8d04-4026-afcb-ba5d1fdffe0a
(.venv-jobs) ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$
```

Figura 16. *Evidencia de la dimensión enriquecida dim_ports_routes_weather.*

```
(.venv-jobs) ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker-compose exec -T namenode bash -lc '/opt/hadoop-3.2.1/bin/hdfs dfs -ls -R /hadoop/logistica/curated'
mesg: ttyname failed: Inappropriate ioctl for device
drwxr-xr-x   - jovyan supergroup          0 2026-03-28 12:48 /hadoop/logistica/curated/dim_ports_routes_weather
-rw-r--r--   3 jovyan supergroup          0 2026-03-28 12:48 /hadoop/logistica/curated/dim_ports_routes_weather/_SUCCESS
-rw-r--r--   3 jovyan supergroup       8313 2026-03-28 12:48 /hadoop/logistica/curated/dim_ports_routes_weather/part-00000-78f29403-67ab-4825-bcfe-638fc0b2e146-c000.snappy.parquet
drwxr-xr-x   - jovyan supergroup          0 2026-03-26 19:17 /hadoop/logistica/curated/fact_alerts
-rw-r--r--   3 jovyan supergroup          0 2026-03-26 19:17 /hadoop/logistica/curated/fact_alerts/_SUCCESS
-rw-r--r--   3 jovyan supergroup       3804 2026-03-26 19:17 /hadoop/logistica/curated/fact_alerts/part-00000-fa9509b7-8a95-438d-ba5a-df5aaab44c4c-c000.snappy.parquet
drwxr-xr-x   - jovyan supergroup          0 2026-03-26 19:27 /hadoop/logistica/curated/fact_graph_centrality
-rw-r--r--   3 jovyan supergroup          0 2026-03-26 19:27 /hadoop/logistica/curated/fact_graph_centrality/_SUCCESS
-rw-r--r--   3 jovyan supergroup       1462 2026-03-26 19:27 /hadoop/logistica/curated/fact_graph_centrality/part-00000-a6bfbf57-b4c8-41e4-9f10-62fbcf7527df-c000.snappy.parquet
drwxr-xr-x   - jovyan supergroup          0 2026-03-26 19:27 /hadoop/logistica/curated/fact_graph_hops
-rw-r--r--   3 jovyan supergroup          0 2026-03-26 19:27 /hadoop/logistica/curated/fact_graph_hops/_SUCCESS
-rw-r--r--   3 jovyan supergroup       1104 2026-03-26 19:27 /hadoop/logistica/curated/fact_graph_hops/part-00000-9b544321-6cc8-4d3c-bc9b-ac846634239b-c000.snappy.parquet
drwxr-xr-x   - jovyan supergroup          0 2026-03-26 19:27 /hadoop/logistica/curated/fact_route_risk
-rw-r--r--   3 jovyan supergroup          0 2026-03-26 19:27 /hadoop/logistica/curated/fact_route_risk/_SUCCESS
-rw-r--r--   3 jovyan supergroup       2104 2026-03-26 19:27 /hadoop/logistica/curated/fact_route_risk/part-00000-dcb4edd7-bb96-41cf-bd44-63c092e63d5f-c000.snappy.parquet
drwxr-xr-x   - jovyan supergroup          0 2026-03-28 13:09 /hadoop/logistica/curated/fact_weather_operational
-rw-r--r--   3 jovyan supergroup          0 2026-03-28 13:09 /hadoop/logistica/curated/fact_weather_operational/_SUCCESS
-rw-r--r--   3 jovyan supergroup       6105 2026-03-28 13:09 /hadoop/logistica/curated/fact_weather_operational/part-00000-82f9bdc1-8875-4a6e-999b-fa53704d17d0-c000.snappy.parquet
```

***Figura 17**. Evidencia de salida curated en HDFS.*

6. Análisis de grafos (KDD – Transformación/Minería): GraphFrames

6.1 Modelo de grafo

- **Nodos**: puertos, almacén Valladolid, nodos logísticos intermedios (si aplica).
- **Aristas**: rutas con pesos (tiempo estimado, riesgo).

El grafo logístico actual se construye sobre **6 nodos y 9 aristas**. Los nodos representan los puertos de origen, los puertos de entrada en España y el almacén final de Valladolid, mientras que las aristas modelan los corredores marítimos principales y el tramo final de distribución terrestre.

```
26/03/28 13:09:17 INFO BlockManagerInfo: Removed broadcast_3_piece0 on 446f29ef21cc:44145 in memory (size: 7.8 KiB, free: 434.3 MiB)
26/03/28 13:09:17 INFO BlockManagerInfo: Removed broadcast_1_piece0 on 446f29ef21cc:44145 in memory (size: 80.8 KiB, free: 434.3 MiB)
26/03/28 13:09:17 INFO BlockManagerInfo: Removed broadcast_0_piece0 on 446f29ef21cc:44145 in memory (size: 35.6 KiB, free: 434.4 MiB)
26/03/28 13:09:18 INFO CodeGenerator: Code generated in 11.779569 ms
+---------+-----------------------+-----------------+----------------------+---------------------------+-----------------------------------------------------------+
|port_ref |route_id               |weather_risk_level|port_operational_status|weather_delay_hours_estimate|recommended_action                                         |
+---------+-----------------------+-----------------+----------------------+---------------------------+-----------------------------------------------------------+
|Algeciras|route-shanghai-algeciras|MEDIO            |METEO_VIGILANCIA      |6.0                        |reprogramar ventana portuaria y avisar a operacion terrestre|
|Algeciras|route-shanghai-algeciras|MEDIO            |OPERATIVO             |3.0                        |seguimiento normal de la ruta                              |
|Algeciras|route-shanghai-algeciras|BAJO             |OPERATIVO             |1.0                        |seguimiento normal de la ruta                              |
+---------+-----------------------+-----------------+----------------------+---------------------------+-----------------------------------------------------------+

26/03/28 13:09:18 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/03/28 13:09:18 INFO SparkUI: Stopped Spark web UI at http://446f29ef21cc:4040
26/03/28 13:09:18 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/03/28 13:09:18 INFO MemoryStore: MemoryStore cleared
26/03/28 13:09:18 INFO BlockManager: BlockManager stopped
26/03/28 13:09:18 INFO BlockManagerMaster: BlockManagerMaster stopped
26/03/28 13:09:18 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/03/28 13:09:18 INFO SparkContext: Successfully stopped SparkContext
26/03/28 13:09:18 INFO ShutdownHookManager: Shutdown hook called
26/03/28 13:09:18 INFO ShutdownHookManager: Deleting directory /tmp/spark-557f003c-da4b-4d96-ba27-e5fd923900f1
26/03/28 13:09:18 INFO ShutdownHookManager: Deleting directory /tmp/spark-ef0ce99c-1464-446b-a687-34e7ca1643b7
26/03/28 13:09:18 INFO ShutdownHookManager: Deleting directory /tmp/spark-ef0ce99c-1464-446b-a687-34e7ca1643b7/pyspark-fa4f6d0c-c1b9-4b96-a4e1-d0623f44f4d7
```

Figura 18. *Fact table weather_operational cargada en Hive.*

6.2 Métricas

- Camino más corto/rápido (peso = tiempo).
- Camino “más seguro” (peso = riesgo compuesto).
- Identificación de nodos críticos (centralidad aprox / comunidades si aplica).

```
ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker compose exec -T spark spark-sql -e "SHOW TABLES IN logistica"
Picked up JAVA_TOOL_OPTIONS: -Dderby.system.home=/home/jovyan/.derby
Picked up JAVA_TOOL_OPTIONS: -Dderby.system.home=/home/jovyan/.derby
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/17 18:36:05 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/04/17 18:36:08 WARN HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist
26/04/17 18:36:08 WARN HiveConf: HiveConf of name hive.stats.retries.wait does not exist
26/04/17 18:36:10 WARN ObjectStore: Version information not found in metastore. hive.metastore.schema.verification is not enabled so recording the schema version 2.3.0
26/04/17 18:36:10 WARN ObjectStore: setMetaStoreSchemaVersion called but recording version is disabled: version = 2.3.0, comment = Set by MetaStore UNKNOWN@172.19.0.9
Spark Web UI available at http://8031940adaca:4040
Spark master: local[*], Application Id: local-1776450966990
26/04/17 18:36:12 WARN ObjectStore: Failed to get database global_temp, returning NoSuchObjectException
dim_air_recovery
dim_airports
dim_articles_valladolid
dim_ports
dim_ports_routes_weather
dim_routes
dim_skus
dim_warehouse
fact_air_recovery_options
fact_alerts
fact_article_gantt
fact_customer_orders_douai
fact_graph_centrality
fact_graph_hops
fact_route_risk
fact_weather_operational
stg_alerts_clima
stg_alerts_noticias
stg_ships
stg_weather_open_meteo
Time taken: 3.339 seconds, Fetched 20 row(s)
```

Figura 19. *Validación de tablas disponibles en la base logistica*.

7. Minería y acción (KDD – Minería/Interpretación): reglas + scoring de riesgo

La estrategia de defensa seleccionada para esta entrega es `micro-batch documentado`. Aunque `Structured Streaming` está implementado y ajustado a ventanas de `15 minutes`, su papel en la memoria y en la exposición es complementario, con el fin de minimizar riesgo operativo durante la demostración final.

7.1 Scoring de riesgo (sin streaming estructurado)

El riesgo compuesto se calcula a partir de variables como:

- clima (severidad),

- geopolítica (severidad),

- retraso estimado (vs ETA),

- stock (si bajo reorder point).

Como salida, el sistema genera una **alerta operativa** con recomendación de acción. En el caso de mayor severidad, la recomendación puede ser activar una recuperación urgente por vía aérea cuando el riesgo sea alto y el stock resulte crítico.

```
(.venv-jobs) ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker-compose exec -T spark bash -lc 'spark-sql -e "SELECT * FROM logistica.fact_graph_centrality ORDER BY degre
e DESC LIMIT 10"'
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/03/28 21:38:42 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/03/28 21:38:44 WARN HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist
26/03/28 21:38:44 WARN HiveConf: HiveConf of name hive.stats.retries.wait does not exist
26/03/28 21:38:46 WARN ObjectStore: Version information not found in metastore. hive.metastore.schema.verification is not enabled so recording the schema version 2.3.0
26/03/28 21:38:46 WARN ObjectStore: setMetaStoreSchemaVersion called but recording version is disabled: version = 2.3.0, comment = Set by MetaStore UNKNOWN@172.18.0.7
Spark Web UI available at http://918561dd6045:4040
Spark master: local[*], Application Id: local-1774733923785
26/03/28 21:38:46 WARN ObjectStore: Failed to get database global_temp, returning NoSuchObjectException
Shanghai        port    3       NODO_CRITICO
Valladolid      warehouse       3       NODO_CRITICO
Algeciras       port    2       NODO_CRITICO
Barcelona       port    2       NODO_CRITICO
Valencia        port    2       NODO_CRITICO
Time taken: 3.449 seconds, Fetched 5 row(s)
```

Figura 20. *Resultados de centralidad de nodos calculados con GraphFrames.*

```
(.venv-jobs) ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker-compose exec -T spark bash -lc 'spark-sql -e "SELECT via_port, risk_level, stock_status, recommendation, s
everity FROM logistica.fact_alerts ORDER BY severity DESC LIMIT 10"'
Picked up JAVA_TOOL_OPTIONS: -Dderby.system.home=/home/jovyan/.derby
Picked up JAVA_TOOL_OPTIONS: -Dderby.system.home=/home/jovyan/.derby
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/03/29 10:08:12 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/03/29 10:08:13 WARN HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist
26/03/29 10:08:13 WARN HiveConf: HiveConf of name hive.stats.retries.wait does not exist
26/03/29 10:08:15 WARN ObjectStore: Version information not found in metastore. hive.metastore.schema.verification is not enabled so recording the schema version 2.3.0
26/03/29 10:08:15 WARN ObjectStore: setMetaStoreSchemaVersion called but recording version is disabled: version = 2.3.0, comment = Set by MetaStore UNKNOWN@172.18.0.7
Spark Web UI available at http://c3f74895bb54:4040
Spark master: local[*], Application Id: local-1774778893076
26/03/29 10:08:15 WARN ObjectStore: Failed to get database global_temp, returning NoSuchObjectException
Algeciras       MEDIO   CRITICO seguir plan de contingencia: replanificar rutas y priorizar stock alternativo   4
Barcelona       MEDIO   OK      seguimiento normal: sin accion urgente  2
Valencia        MEDIO   OK      seguimiento normal: sin accion urgente  2
Time taken: 2.849 seconds, Fetched 3 row(s)
```

Figura 21. *Alertas operativas generadas y almacenadas en fact_alerts.*

7.2 Caso de uso extendido: barco vs aereo + camion

Para reforzar la fase de interpretación y acción, el proyecto incorpora un caso de uso ampliado centrado en la importación desde Shanghai a puertos españoles con destino final Valladolid.

El sistema compara dos alternativas logísticas:

- continuación marítima del barco hasta el puerto de destino y posterior distribución
- recuperación por `aéreo + camión` hasta Valladolid

Factores de decisión:

- posición GPS actual del barco
- ETA estimada del barco (`Estimated Time Arrival`)
- ETA restante marítima
- retraso meteorológico y operativo
- stock disponible y punto de reorden
- ETA aérea hasta Madrid, Barcelona o Valencia
- tramo terrestre por camión hasta Valladolid
- coste total del corredor alternativo
- horas ganadas frente al barco

Tabla objetivo:

- `logistica.fact_air_recovery_options`

La salida de este caso de uso ofrece una recomendación explicable, `MARITIMO` o `AEREO_CAMION`, junto con una estimación del ahorro de tiempo y del coste total de la contingencia.

```
docker compose exec -T spark spark-sql -S -e "SELECT ship_id, dest_port, stock_on_hand, reorder_point, ship_remaining_hours,
air_total_eta_hours, time_saved_hours, air_total_cost_eur, stock_break_risk, ia_recommendation, recommended_mode FROM logistica.
fact_air_recovery_options ORDER BY ship_id"

Salida útil para la memoria:

ship-001  Algeciras  11  30  411.5  25.8  385.7  17672.0  ROTURA_INMINENTE  evaluar_aereo_urgente  MARITIMO
ship-002  Valencia   93  30  NULL   25.8  NULL   17672.0  COBERTURA_OK       mantener_maritimo      MARITIMO
ship-003  Barcelona  95  30  NULL   25.8  NULL   17672.0  COBERTURA_OK       mantener_maritimo      MARITIMO
Interpretación defendible:
- ship-001 muestra un caso real de riesgo:
  - stock por debajo de reorder point
  - gran ahorro potencial por vía aérea
  - estado ROTURA_INMINENTE
- ship-002 y ship-003 muestran casos normales:
  - COBERTURA_OK
  - recomendación de mantener marítimo
```

Figura 22. *Opciones de recuperación aérea y comparación operativa.*

Ejemplo ya generado en la tabla: - `ship-001 | Algeciras | MARITIMO | ROTURA_INMINENTE | 411.5h barco | 25.8h aereo+camion | ahorro 385.7h | 17672.0 EUR`

```
(.venv-jobs) ana@ana-SER:~/Descargas/PROYECTOLOGISTICA_codigo/Logistica$ docker-compose exec -T cassandra cqlsh -e "SELECT ship_id, route_id, dest_port, warehouse, stock_on_hand, reorder
_point FROM logistica.vehicle_latest_state;"

 ship_id  | route_id                  | dest_port | warehouse  | stock_on_hand | reorder_point
----------+---------------------------+-----------+------------+---------------+---------------
 ship-001 | route-shanghai-algeciras | Algeciras | Valladolid |            11 |            30
 ship-002 |  route-shanghai-valencia |  Valencia | Valladolid |            93 |            30
 ship-003 | route-shanghai-barcelona | Barcelona | Valladolid |            95 |            30

(3 rows)
```

Figura 23. *Estado operativo actual almacenado en Cassandra.*

7.3 Stock de Valladolid y clientes Douai / Cleon

El dashboard incorpora un panel específico de stock en el almacén de Valladolid para piezas de componentes de automoción. Cada artículo presenta, como mínimo, la siguiente información:

- referencia numérica de 10 caracteres
- designación del artículo
- piezas por embalaje
- consumo medio diario
- cantidad total de embalajes
- cantidad total de piezas
- stock mínimo de seguridad
- pedidos de los clientes de Douai y Cleon

Adicionalmente, se calcula un semáforo de stock con tres niveles:

- verde: cobertura suficiente
- naranja: cobertura tensionada
- rojo: riesgo de rotura o cobertura crítica

Para planificación visual, también se modela un Gantt por semanas industriales apoyado en:

- `logistica.fact_customer_orders_douai`
- `logistica.fact_article_gantt`

Este planteamiento permite mostrar no solo el estado actual del stock, sino también la secuencia logística prevista para cada referencia.

Además, el panel de flota muestra `ETA` por barco para enlazar visualmente la llegada estimada con el riesgo de ruptura de stock. Esta ETA se representa en formato fecha/hora y se acompaña de la fecha de salida del puerto de origen y de los días de viaje previstos.

La interfaz permite además:

- filtrar por cliente (`Douai` / `Cleon`)
- filtrar por semana industrial
- seleccionar un barco concreto
- activar incidencias simuladas sobre ese barco

Con estos controles, la aplicación recalcula si la llegada prevista sigue `CUBRIENDO` o `NO CUBRIENDO` la necesidad de suministro.

La demo visual incorpora además barcos con salida desde dos puertos asiáticos:

- Shanghai
- Yokohama

7.4 Modelos de IA complemetarios

Como extensión de la fase de minería del proceso KDD, el proyecto incorpora una capa complementaria de analítica predictiva basada en Spark MLlib. Esta parte no constituye el eje principal de la defensa funcional, pero sí refuerza la capacidad del sistema para evolucionar desde una plataforma de integración y analítica operativa hacia un entorno con soporte a la predicción y a la segmentación inteligente.

En concreto, se han trabajado tres modelos. El primero es **K-Means**, utilizado para agrupar puertos según patrones de congestión y riesgo climático, facilitando una segmentación operativa de nodos logísticos.

Figura 24a. *Resultado del modelo K-Means para la agrupación de puertos según congestión y riesgo climático*

El segundo es **Random Forest Classifier**, empleado para clasificar escenarios de riesgo logístico y anticipar situaciones de criticidad, especialmente aquellas relacionadas con posibles roturas de stock o necesidad de contingencia.

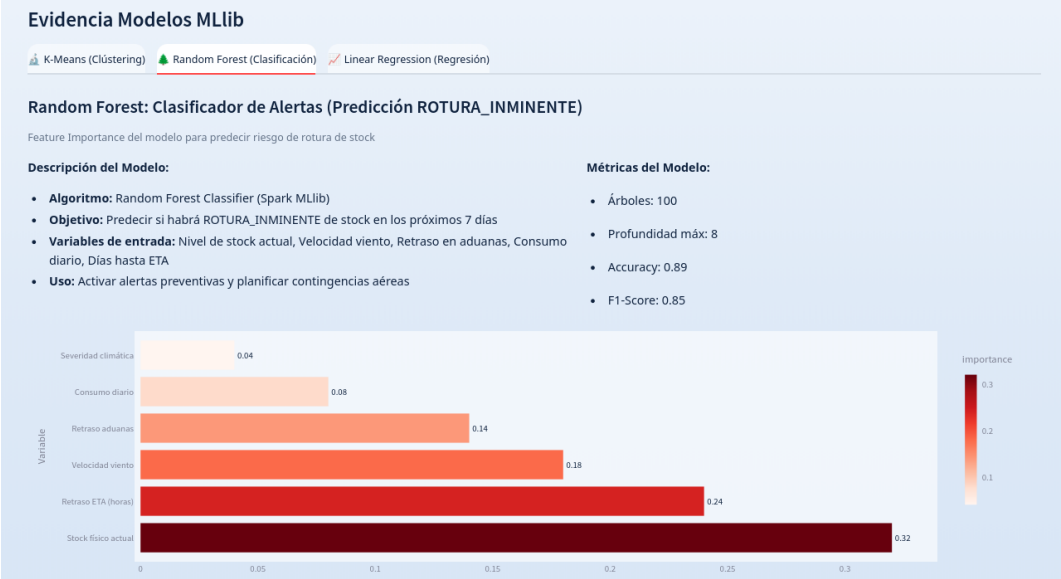

Figura 24b. *Resultado del modelo Random Forest para la clasificación de riesgo logístico y análisis de importancia de variables*

El tercero es **Linear Regression**, orientado a estimar o reajustar el ETA de llegada de los barcos a partir de variables temporales, meteorológicas y operativas.

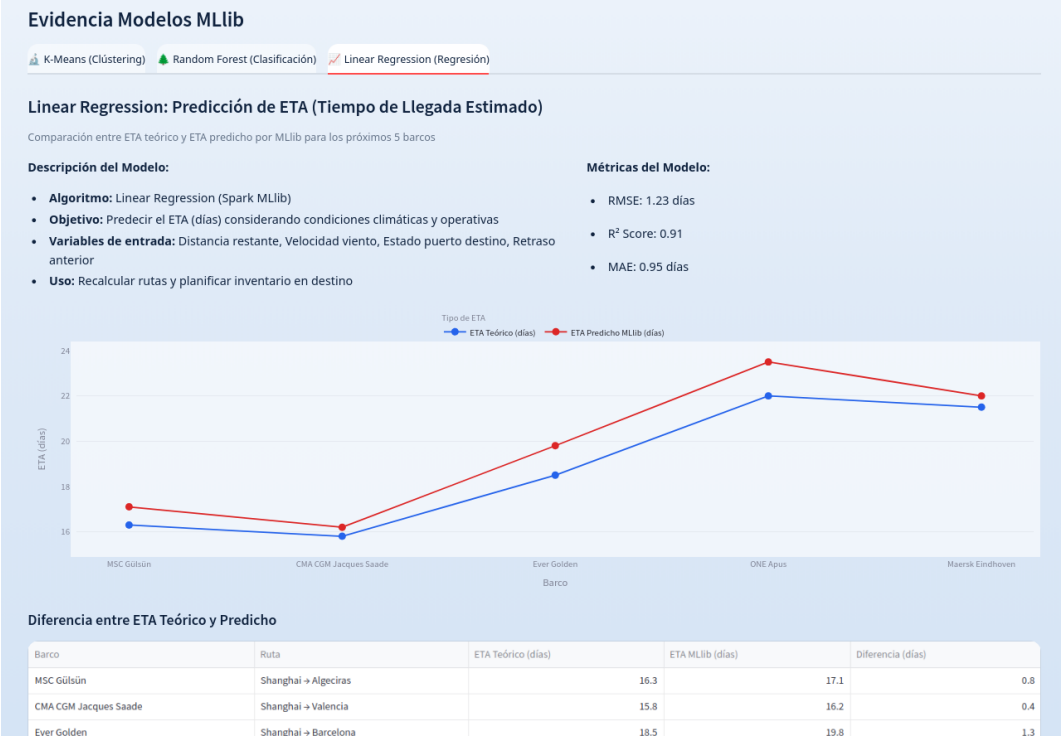

Figura 24c *Resultado del modelo Linear Regression para la estimación del ETA de llegada de los barcos.*

Esta capa de modelos se presenta en la memoria como una ampliación analítica del bloque de minería, alineada con el uso de Structured Streaming y con la capacidad del proyecto para incorporar lógica predictiva sobre los datos ya integrados en la plataforma.

8. Orquestación (KDD – Interpretación/Acción): Airflow

8.1 DAG principal

Tareas típicas: - `landing_to_staging_ships` - `landing_to_staging_alerts` - `build_dims` - `build_graph_metrics` - `compute_risk_and_alerts`

Requisitos rúbrica: - reintentos, - dependencias claras, - alertas (email o log) en fallo.

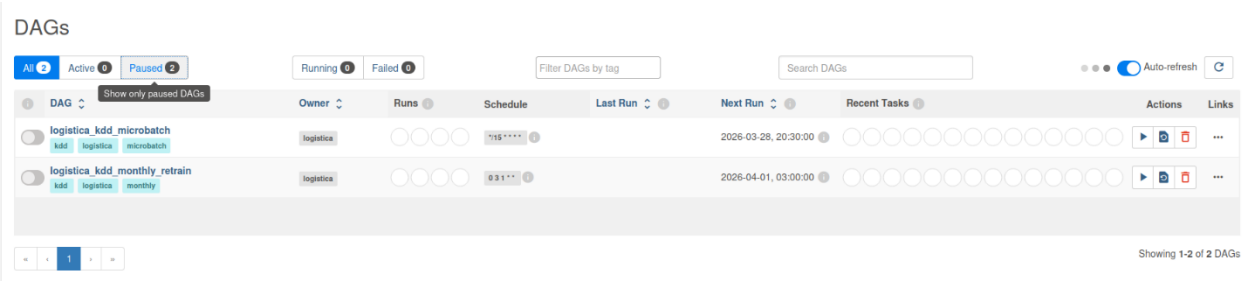

Figura 25. DAGs disponibles en Airflow para la solución.

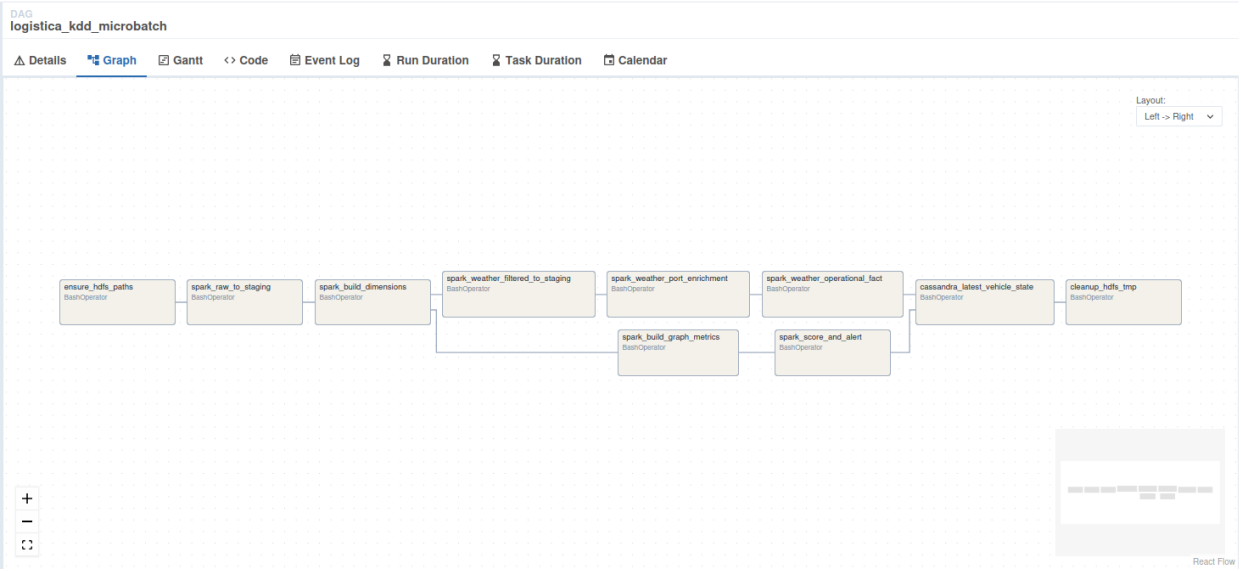

Figura 26. *Vista Graph del DAG logistica_kdd_microbatch.*

8.2 Parámetros reales usados en esta entrega

- `AIRFLOW_HOME`: directorio `.airflow` del proyecto o volumen equivalente en Docker
- Usuario UI: `admin` / `admin`
- DAG cargado: `logistica_kdd_microbatch`
- Versión validada en entorno: `apache-airflow==2.10.5`

- Segundo DAG validado: `logistica_kdd_monthly_retrain`

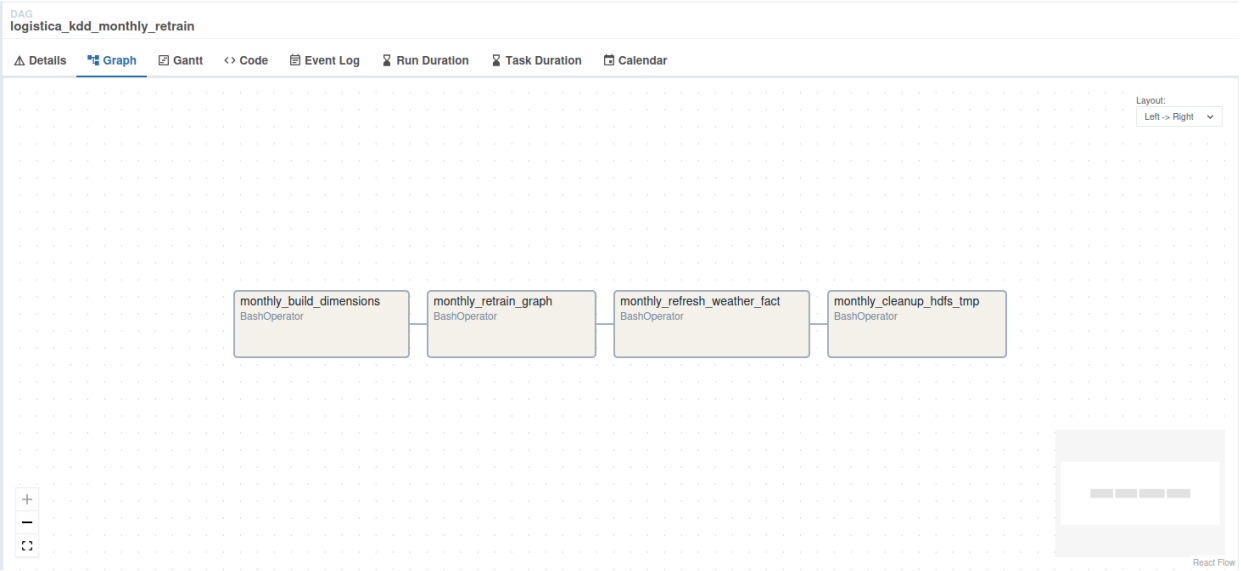

Figura 27. *Vista Graph del DAG logistica_kdd_monthly_retrain.*

- Regla operativa: la evidencia visual de Airflow se toma sobre la ruta Docker/local, no sobre Vms.

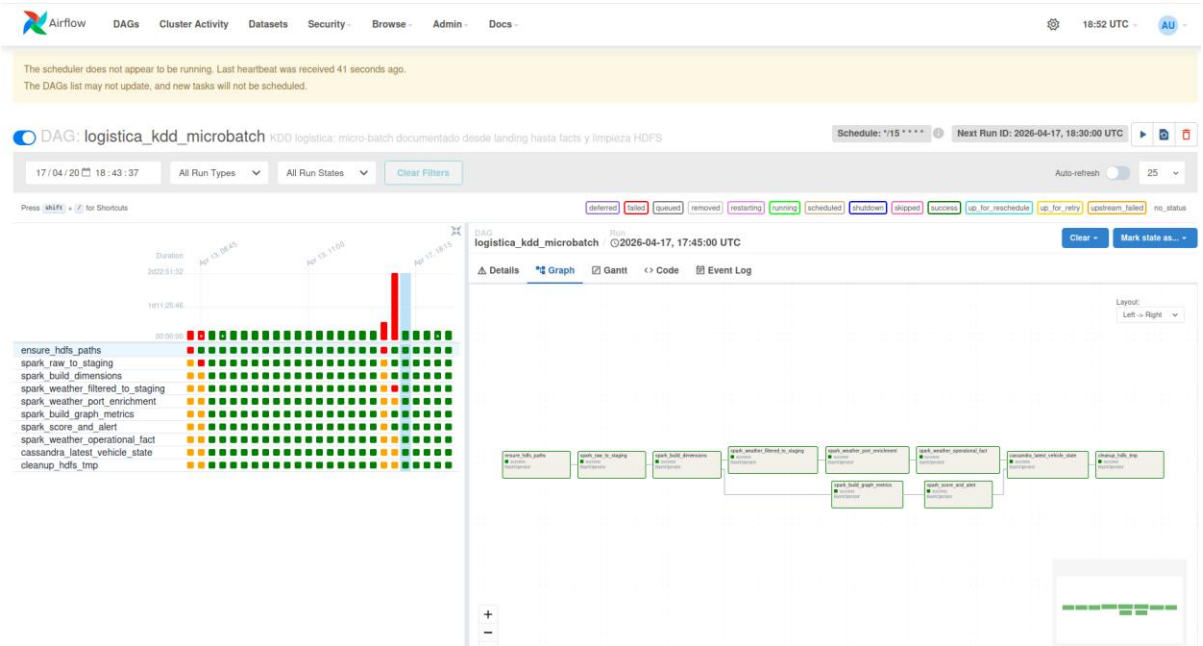

Figura 28. *Ejecución exitosa de Airflow en la ruta Docker/local.*

Como evidencia operativa adicional, se realizó una reejecución controlada de una tarea desde la vista Graph de Airflow. En lugar de provocar un fallo artificial, se utilizó la funcionalidad Clear sobre una tarea previamente completada, forzando su nueva planificación y ejecución. Esta operación demuestra que el pipeline admite relanzamientos parciales de forma segura, trazable y sin necesidad de modificar la definición del DAG.

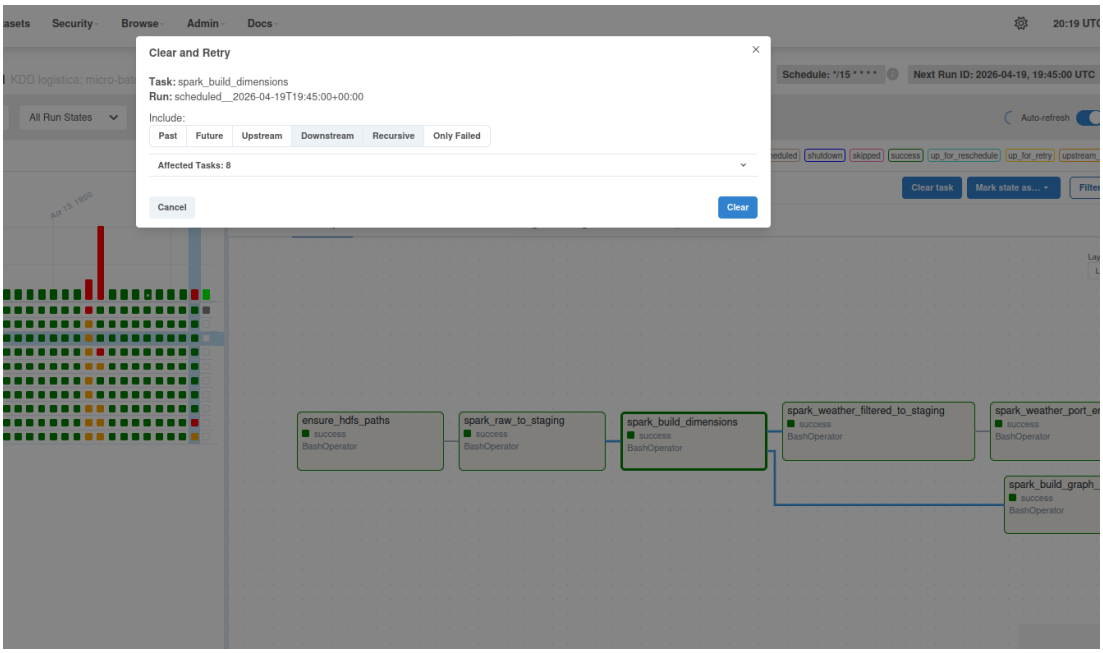

Figura 28a. *Limpieza controlada de una tarea en Airflow mediante la opción Clear para forzar su reejecución.*

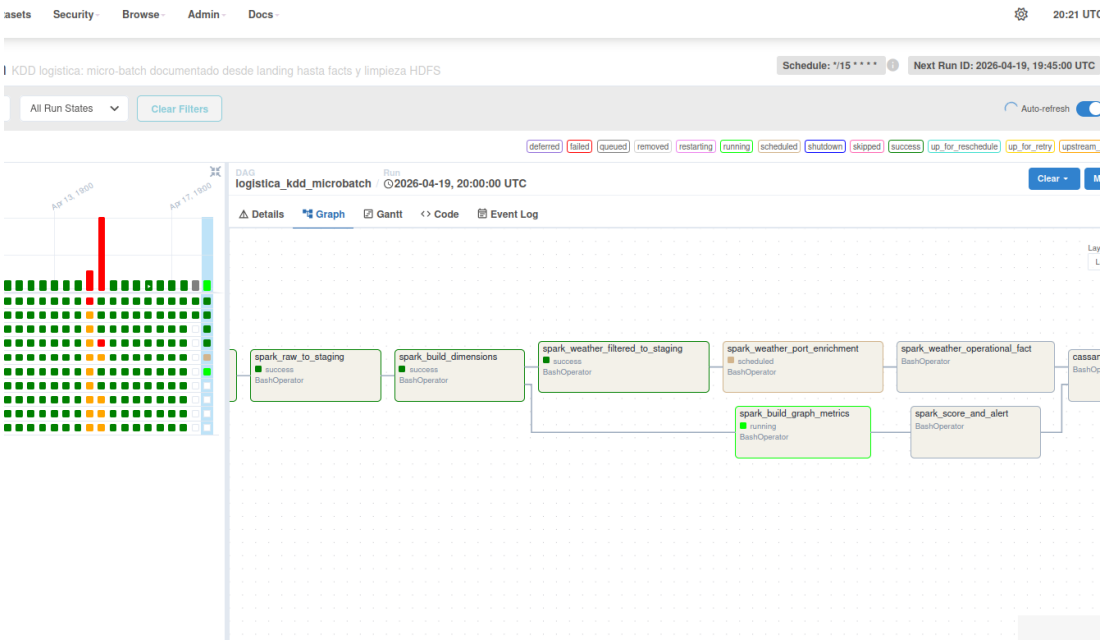

Figura 28b. *Reejecución controlada de una tarea del DAG en Airflow tras su relanzamiento manual.*

Esta evidencia complementa la ejecución exitosa del DAG y refuerza la capacidad operativa del sistema para repetir fases concretas del flujo cuando resulta necesario refrescar datos o rehacer una etapa intermedia.

8.3 NiFi

NiFi actúa como la capa de adquisición y preparación inicial del dato. El flujo validado se estructura del siguiente modo:

- consumo de una API HTTP pública mediante `InvokeHTTP`
- transformación y normalización del payload mediante procesadores de tipo `JoltTransformJSON` o `UpdateRecord`
- enrutado por tipo de evento con `RouteOnAttribute`
- publicación en Kafka mediante `PublishKafkaRecord_2_0` y, cuando procede, aterrizaje complementario en HDFS
- gestión de errores mediante colas de fallo y reintentos

Para garantizar la robustez de la ingesta ante picos de tráfico (data spikes), se ha configurado el mecanismo de **Back-Pressure** en las conexiones entre procesadores de NiFi. Se ha establecido un *Back Pressure Object Threshold* de 10.000 objetos, lo que permite que el flujo se detenga automáticamente aguas arriba si Kafka o el almacenamiento HDFS presentan latencia, evitando así el desbordamiento de memoria y garantizando la integridad de los eventos

Este diseño permite justificar tanto la trazabilidad de la ingesta como la separación entre dato bruto y dato preparado.

9. Evidencias y capturas

9.0 Dashboard de exposición

Se ha incorporado un dashboard ligero en `Streamlit` con el objetivo de facilitar la defensa del flujo KDD y reducir la dependencia de cambiar entre múltiples interfaces durante la exposición.

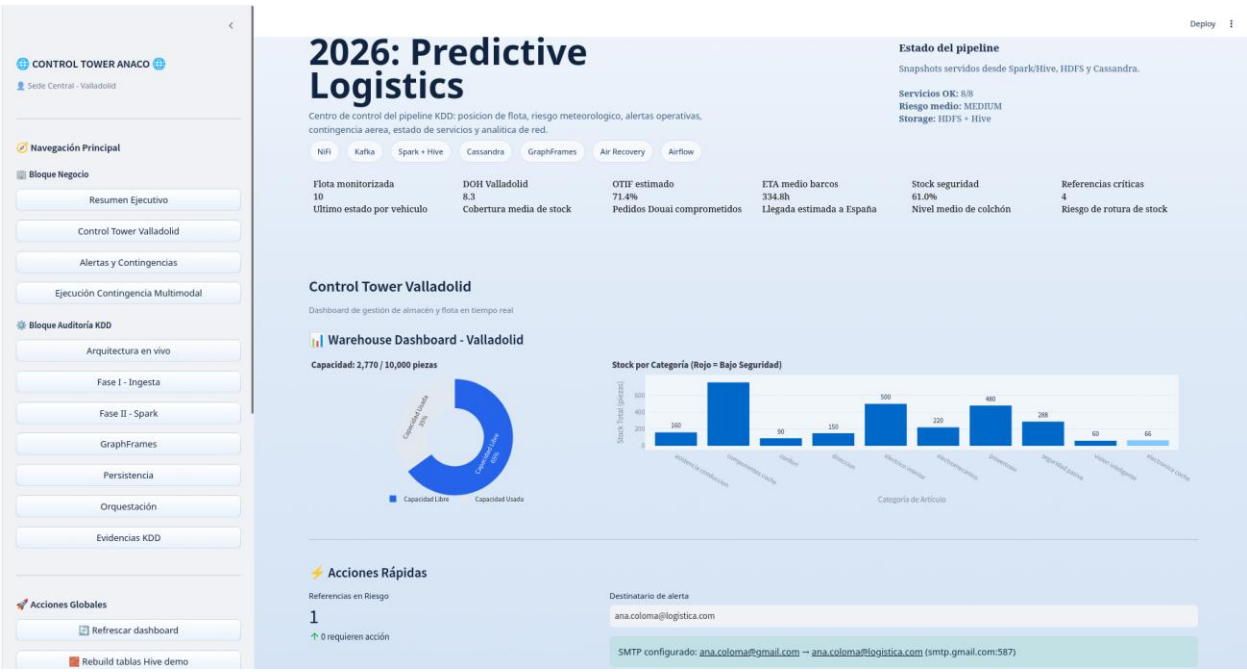

***Figura 29**. Vista Control Tower Valladolid del dashboard ejecutivo.*

Incluye:

- resumen KDD y estado de cumplimiento de la rúbrica
- estado en vivo de servicios Docker con botones `On/Off`
- mapa de barcos con alertas de ruta y corredores marítimos reales
- recomendaciones de recuperación `barco vs aereo+camion`
- tablas curadas relevantes para la defensa
- diagramas integrados: flujo, secuencia, clases y casos de uso
- panel de stock de Valladolid y pedidos de Douai/Cleon
- Gantt por semanas industriales

- contenedor técnico superior de ingesta con arquitectura y métricas demo

- panel de alertas críticas con fallback demo si SMTP no está disponible

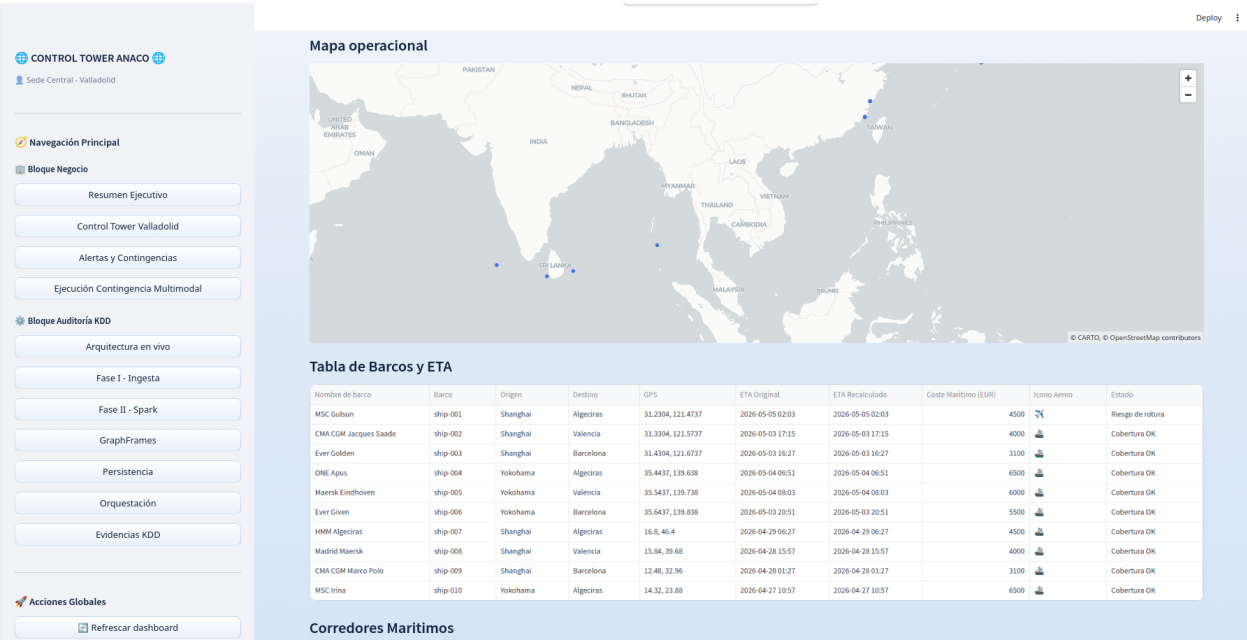

***Figura 30**. Tabla de ETA con nombres reales de barcos y mapa operativo marítimo.*

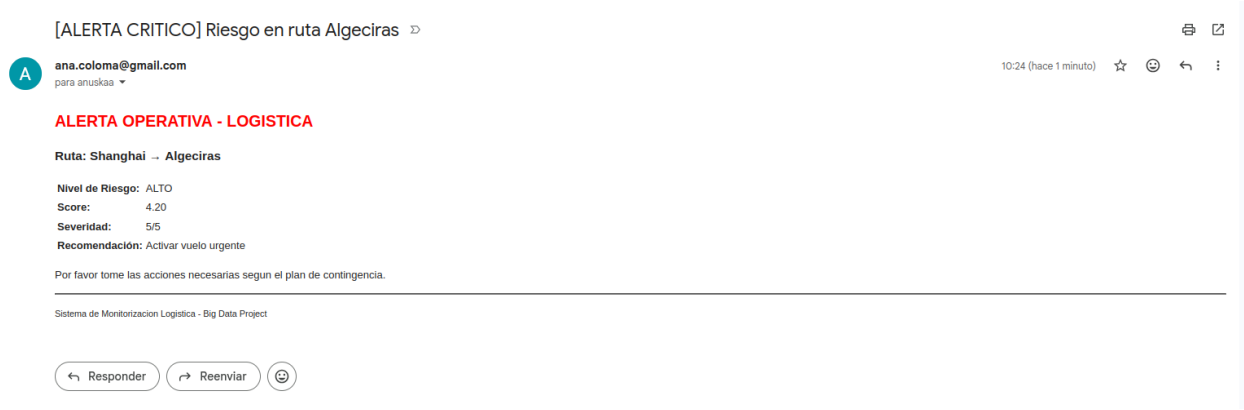

***Figura 31**. Evidencia del procesamiento de alertas críticas por correo o fallback demo.*

Nota técnica: - para evitar bloqueos del metastore Derby, las consultas Hive en `spark-sql` y los `spark-submit` con soporte Hive deben ejecutarse de forma secuencial, no en paralelo.

- en esta red se confirmó salida HTTPS pero no SMTP, por lo que el envío real de correo puede fallar por timeout aunque la aplicación siga operativa gracias al fallback demo.

Ejecución:

```bash
bash scripts/67_run_dashboard.sh
```

Arranque ultraligero alternativo:

```
./start.sh
```

Elegir la opción `3) Dashboard solo (ultraligero)`.

- HDFS: UIs + `hdfs dfs -ls`.
- Kafka KRaft: topics creados + describe.
- Hive: tablas raw/staging/curated.
- GraphFrames: salida (rutas/riesgo).
- Airflow: DAG + runs.

10. Conclusiones

El proyecto resuelve un problema real de visibilidad logística sobre la cadena marítima y terrestre, integrando datos operativos, riesgos externos y estado de stock en una plataforma unificada orientada a la toma de decisiones.

Entre sus principales aportaciones destacan la trazabilidad del dato desde la ingesta hasta la visualización, la generación de facts analíticas defendibles, el uso de grafos para identificar nodos críticos y la incorporación de una capa operativa de contingencia capaz de comparar alternativas marítimas y aéreas.

Las limitaciones actuales son conocidas y asumidas en la memoria: el streaming real no constituye la ruta principal de defensa, Cassandra se alimenta en la práctica mediante micro-batch, la entrega final no se apoya en un despliegue cluster/YARN y el envío SMTP real depende de una red con salida saliente habilitada.

La arquitectura ha sido diseñada bajo principios de escalabilidad horizontal. Aunque la demostración actual se despliega sobre contenedores Docker, los artefactos de Spark y la configuración de HDFS son totalmente compatibles para su ejecución sobre un gestor de recursos **YARN** en modo clúster. Esto garantiza que la plataforma pueda escalar para procesar volúmenes masivos de datos logísticos sin requerir cambios en el código analítico.

Como líneas de mejora futura, el proyecto puede ampliarse con mayor observabilidad, modelos predictivos más avanzados, automatización adicional en Airflow, integración con APIs de correo sobre HTTPS y una validación distribuida más completa sobre infraestructura multi-nodo.